

ESERCIZI PER CASA:

```
{ int a = 0; y = 1;
```

⊛

```
void Exec() { int x;
```

⊛⊛

```
y = fun();
```

```
a = a + 1;
```

```
}
```

```
while (a < 1) { exec(); }
```

① Cosa devo definire: ~~fun()~~ fun()

x non è Inizializzata ma nemmeno Usata Quindi è OK

Dobbiamo Utilizzare una Variabile non locale che abbia sia un Ambiente a Tempo di definizione sia nel Blocco di Chiamata

Bisogna Sempre Giustificare!

Se possibile MAI Usare la Variabile del ciclo

Salgo x visto che non ha Ancora Nessuno scopo nel Codice.

Blocco Di DEFINIZIONE

```
int x = 10;  
int fun() { return x; }
```



RIFERIMENTO AD UNA VARIABILE NON GLOBALE,
Quindi devo sfruttare le REGOLE DI SCOPING

Blocco Di CHIAMATA

x = a;

Deve Cambiare Nelle Esecuzioni, Quindi
Sfrutto la Variabile del ciclo

SCOPING STATICO: fun() ha come AMBIENTE Quello Globale e
Restituisce Sempre 10

SCOPING DINAMICO: fun() ha come AMBIENTE Quello dentro EXEC
dove $x = a$ per Ogni Chiamata e Quindi

☞ Vale 0 alla prima Chiamata

☞ Vale 1 alla SECONDA

Catena Statica/Dinamica un Altro Esercizio possibile

C'è Sempre la DEF. di Scoping Statico & Dinamico e la
differenza

BINDING:

SHALLOW

Si Riferisce all' AMBIENTE Quando Viene Effettuata la Chiamata

DEEP

Si Riferisce all' AMBIENTE dove si CREA il legame

Statico

Dinamico

Quando Si parla di BINDING? Definire lo SCOPING.

Quando una PROCEDURA ha un AMBIENTE non locale, Servono Anche Quelle di BINDING Quando la procedura con AMBIENTE NON locale è passata Come PARAMETRO ad un Altra PROCEDURA

```
int x=3, y=2; ①
```

```
int fun1(int n){ return x+y+n; }
```

→ Mi SERVE Scoping

```
int fun2(int h(int B)) {
```

```
    int x=4, y=4; ③ AMBIENTE di Chiamata con PARAMETRO ATTUALE  
    return h(5)+x
```

```
}
```

```
⋮
```

```
{ int x=6, y=3; z=fun2(fun1);
```

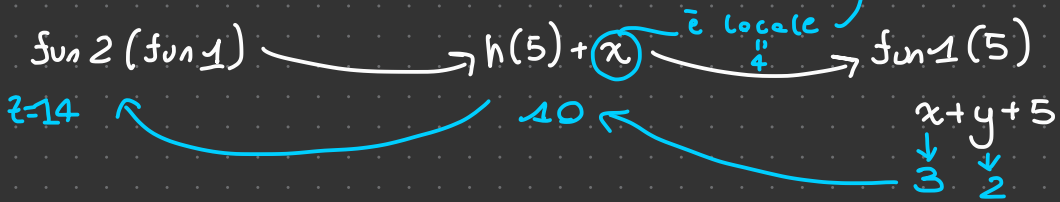
→ Mi Serve BINDING

② AMBIENTE dove viene CREATO il legame

② AMBIENTE D. DEFINIZIONE

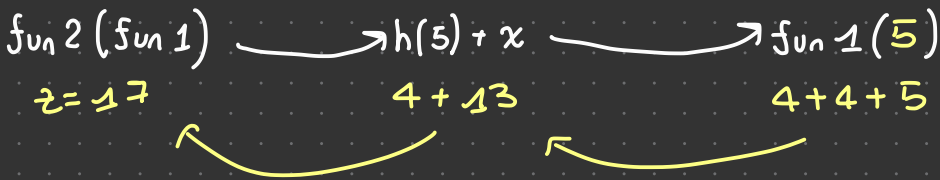
Procediamo per casi:

SCOPING STATICO Viene RESTITUITO $z=14$; visto che ho $x=3$ e $y=2$ (AMBIENTE ①)

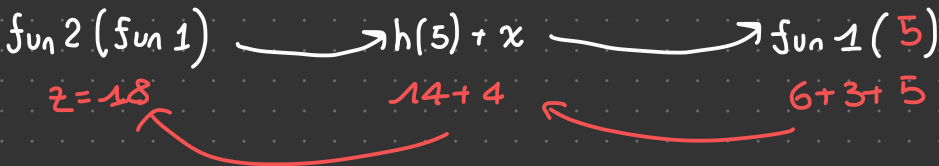


SCOPING DINAMICO:

① SHALLOW $\Rightarrow z = 17$; (AMBIENTE ③)



② DEEP $\Rightarrow z = 18$; (AMBIENTE ②)



SCOPING IMPLEMENTAZIONE:

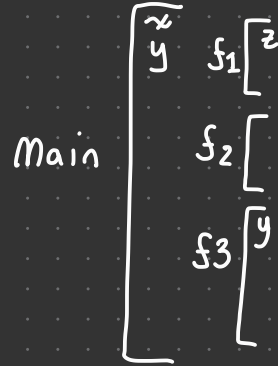
Sempre DEFINIRE Concetti di SCOPING, cos'è Link Statico e anche la CRT (definizione e com'è Aggiornata/letta)

```
int x, y;
```

```
void f1() { int z = x + y; }
```

```
void f2() { y = y + 1; }
```

```
void f3() { int y = x + 2;
           f2();
           f1();
           }
```



```
x = 5, y = 2;
```

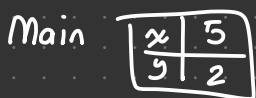
```
fun3();
```

$Sd(Main) = 0$

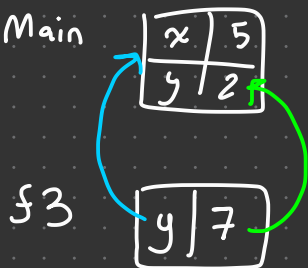
$Sd(f1) = Sd(f2) = Sd(f3) = 1$

CATENA DELLE CHIAMATE $\Rightarrow f3 \rightarrow f2 \rightarrow f1$

SCOPING STATICO:



Chiamata a f3()



$$sd(chiamante) - sd(chiamato) + 1$$

$$K_{f3} = sd(Main) - sd(f3) + 1 = 0$$

$$CS(f3) = Main$$

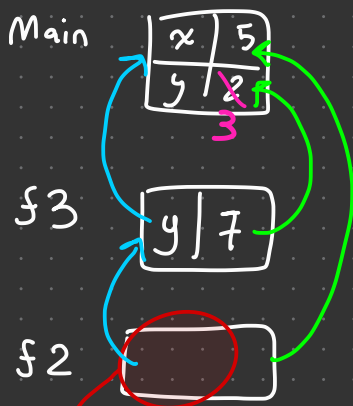
$$y = x + 2 = 7$$

Devo risolverla:

$$Nx = 5 \leftarrow Nx = sd(f3) - sd(Main) = 1$$

$$sd(chi la usa) - sd(piu vicina che la def.)$$

Chiamata f3 a f2



$$K_{f2} = sd(f3) - sd(f2) + 1 = 1$$

$$CS(f2) = CS(f3) = Main$$

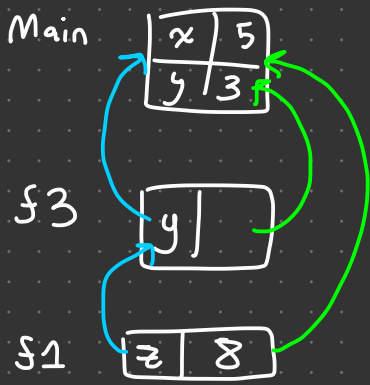
$$y = y + 1 = 3$$

non è locale:

$$Ny = sd(f3) - sd(main) = 1$$

Risalgo di 1 volta la Catena Statica

Ci vanno solo quelle dichiarate e BASTA !!!



$$K_{f_1} = Sd(f_3) - Sd(f_1) + 1 = 1$$

$$CS(f_1) = CS(f_2) = \text{Main}$$

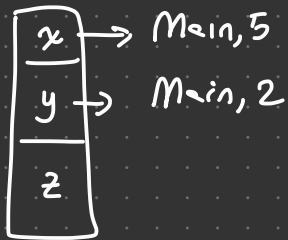
$z = x + y = 8$

$N_y = Sd(f_1) - Sd(\text{Main}) = 1$

$\Rightarrow N_x = Sd(f_1) - Sd(\text{Main}) = 1$

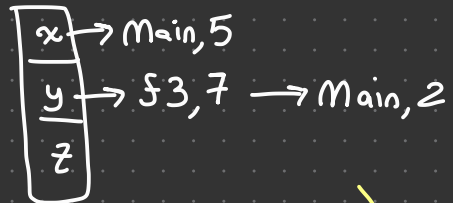
Poi Via Via Si distruggono le Chiamate

la SCOPING DINAMICO:



Eseguo Main

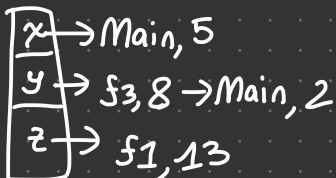
Chiamo
f3()



Eseguo f3()

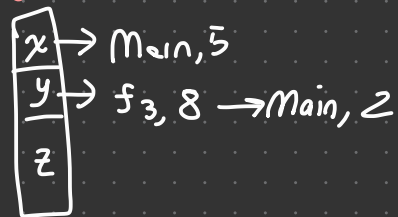
Chiamo
f2

Eseguo f1



Eseguo f2

Chiamo
f1()



Induzione Naturale NON c'è Ma c'è Solo Quelle

STRUTTURALE

PASSAGGIO PER VALORE:

Def. Tipologia di Passaggio dell' Esercizio

Risultato non fatto!

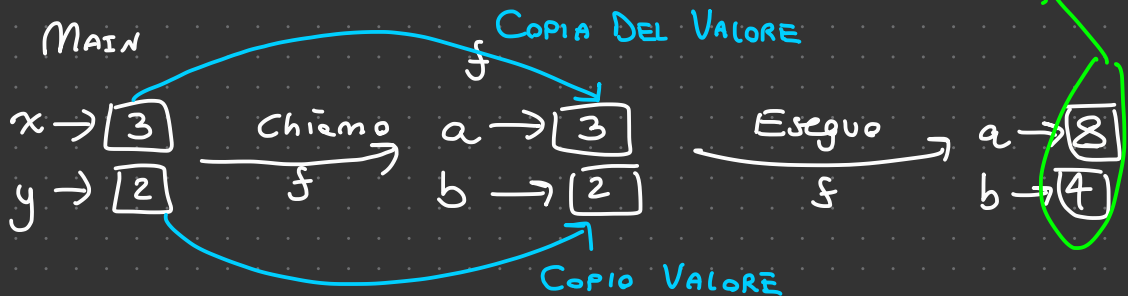
Codice che definisca una funzione e utilizzo:

```
f(int a, int b){  
    a = a + 5;  
    b = 2b;  
}
```

```
main(){  
    int x = 3, y = 2;  
    f(x, y);  
}
```

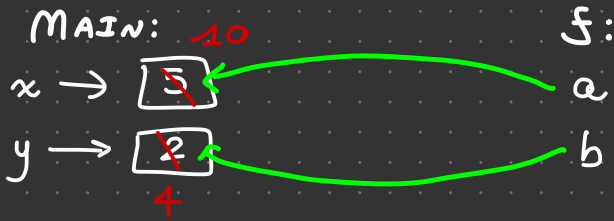
Sovrascritte ALLE VAR
Di Prima (Stesse Loc.)

Cosa Succede in MEMORIA?



Sia a, b Vengono Eliminate Mentre x, y Rimangono INVARIATI

PASSAGGIO PER RIFERIMENTO:



NON Crea Spazio con Valori ma Avrà INDIRIZZI in Modo che puntino alla Stessa cella di x, y

Si CONCLUDE il MAIN con le Variabili x e y con nuovi Valori visto che la MODIFICA Viene Osservata/ha Valore anche ALL'esterno della PROCEDURA (Una Volta che ho Eliminato Gli Indirizzi).

RICORSIONE:

Da Ricorsivo, Ricorsivo In CODA (da DEFINIRE ENTRAMBI)

Mostrare Anche le Sequenze delle Chiamate oltre a dire che Cosa fa

```
int f(lista list){  
    IF len(list)==0 return list;  
    ELSE return (car(list) + function(cdr(list)))
```

SU INPUT (3,6,9)

$f((3,6,9)) \Rightarrow 18$
 $\hookrightarrow 3 + f((6,9)) \leftarrow 15$
 $\hookrightarrow 6 + f((9)) \leftarrow 9$
 $\hookrightarrow 9 + f((\emptyset)) \leftarrow 0$

Questa f.z. Ricorsiva fa la Somma di Tutti Gli Elementi della lista

```
int f(lista list)  
    return SRC(list, 0)
```

Ricordarsi di far PARTIRE Ricorsione di CODA

```
int SRC(lista list, int Res)
```

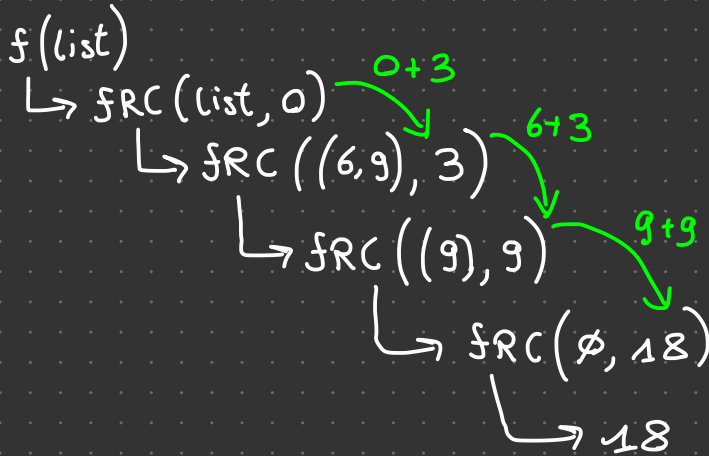
```
    IF len(list)==0
```

```
        return Res;
```

```
    ELSE return SRC(cdr(list, res + car(list)))
```

→ Risultato che MAN MANO mi Costruisco

Spesso il valore del caso BASE è il valore al quale viene Inizializzato il Valore del RISULTATO PARZIALE che costruirò e dov'è RITORNARLO al POSTO del Valore Costante.



SEMANTICA:

$\rho = [x \mapsto 2, y \mapsto 4] \rightarrow$ Va detto cos'è un AMBIENTE

$\delta = [y \mapsto 4] \rightarrow$ def. Mem

↓
Associa ID $\leftrightarrow$ Val

ASSEGNAMENTO: $y := (x + 4) * y$

① Valuto $(x + 4)$ Quindi RECUPERO il Valore di x dall'Ambiente

$$\rho \vdash \langle x + 4, \delta \rangle \xrightarrow{*} 6, \delta$$

$$\rho \vdash \langle (x + 4) * y, \delta \rangle \xrightarrow{*} \langle 6 * y, \delta \rangle$$

$$\rho(x) = 2 \leftarrow$$

② Valuto y :

$$\rho \vdash \langle 6 * y, \delta \rangle \xrightarrow{*} \langle 6 * 4, \delta \rangle$$

$$\rho \vdash \langle 6 * y, \delta \rangle \xrightarrow{*} \langle 24, \delta \rangle$$

$$\rho(y) \text{ e } \delta[y \mapsto 4]$$

③ Eseguo Moltiplicazione

$$\rho \vdash \langle 6 * y, \delta \rangle \xrightarrow{*} \langle 24, \delta \rangle$$

$$\rho \vdash \langle 6 * 4, \delta \rangle \xrightarrow{*}$$

④ Completo Assegnamento Con Elaborazioni Precedenti

$$\rho \vdash \langle y := (x + 4) * y, \delta \rangle \xrightarrow{*} \langle y := 24, \delta \rangle$$

$$\Rightarrow \delta[y \mapsto 24]$$

SEMANTICA STATICA STESSO COMANDO: $\Delta = [x \mapsto \text{int}, y \mapsto \text{int}_{loc}]$

$$\frac{\Delta \vdash (x+4) * y : \text{int}}{\Delta \vdash y := (x+4) * y : \text{int}}, \Delta(y) = \text{Int}_{loc}$$

$$\frac{\Delta \vdash x : \text{int} \quad \vdash 4 : \text{int}}{\Delta \vdash x+4 : \text{int}} \quad \Delta(x) = \text{int} \quad \text{Assioma}$$

$$\frac{}{\Delta \vdash y : \text{int}} \quad \Delta(y) = \text{int}_{loc} \quad \text{Quindi c'è COERENZA di TIPO}$$

C₁ MOLTIPLICAZIONE tra 2 Tipi INTERI Rimane INTERA, Quindi Posso Affermare che il Comando è Ben formato